PyPyrus: Project Plan

Arnór Daníel Moncada

February 23, 2026

Motivation

Machine learning systems are increasingly being adopted and deployed in high-impact domains such as healthcare, finance, and public administration [1, 2, 3]. As a result, regulatory frameworks such as the EU AI Act introduces requirements regarding transparency, traceability, and documentation of training data for high-risk AI systems [4].

Today's ML tooling focuses primarily on experiment tracking, model performance and deployment. Dataset-level traceability during training remains largely unsupported. In particular, there is no standard mechanism for automatically recording which samples were accessed, in what order, and how frequently under stochastic training procedures such as shuffling and batching. This creates challenges for auditing of dataset usage, reproducibility of training runs, and compliance with regulatory logging and reporting obligations. The goal of this thesis is to design and implement `PyPyrus`, a lightweight instrumentation provenance layer that automatically logs data usage during model training and provides a simple interface for reconstructing and reporting dataset access history and other valuable reportable fields. The system will focus on single-machine Pytorch [5] training pipelines.

Background

Machine learning tooling and infrastructure currently provide several partial solutions.

- **Experiment Tracking systems:** Tools such as WandB [6] and MLflow [7] enable logging of training runs, hyperparameters, and artifacts. However these systems typically treat the datasets as versioned artifacts rather than tracking which individual samples were accessed and how often. Though similar granularity can be achieved through a combination of features (e.g. WandB Tables) they require careful thought from the developers and manual discipline to enforce exact logging, which does not guarantee comprehensive capture of runtime data access events.

- **Data Versioning Tools:** Tools such as DVC [8] enable dataset versioning and snapshot management. While very useful for reproducability at file level, they do not capture runtime data access patterns.
- **Dataset Documentation Frameworks:** Approaches such as **SMRS reporting framework** [9] propose a standardized way of documenting models and datasets. However, they are more so reporting frameworks rather than concrete implementations which heavily imply a lot of manual reporting which do not guarantee alignment between documentation and actual training behavior.

In summary, existing solutions operate either at the artifact level, rely on manual documentation, or focus on experiment-level metadata. But there is no lightweight system that can be integrated directly with ML training loops, to automatically capture deterministic, sample-level dataset usage independent of the developer.

Research Question

The overall research question is:

Can dataset-level provenance during machine learning training be automatically captured at sample level granularity in a lightweight manner, that preserves reproducibility while introducing acceptable computational overhead.

To investigate this question, the following sub-questions are considered:

- How can dataset access events be instrumented within standard ML frameworks without requiring intrusive modifications to existing training pipelines?
- To what extent can logged dataset access events be used to reconstruct training data order under data shuffling?
- What is the computational overhead introduced by fine-grained dataset access logging across varying dataset sizes and training configurations?
- Do the generated provenance records meaningfully support traceability requirements outlined in Articles 10–13 of the EU AI Act?

Time Plan

Activity	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12	Week 13	Week 14	Week 15	Week 16	Week 17	Week 18	Week 19	Week 20	Week 21	Week 22
	26/01 - 01/02	01/02 - 08/02	09/02 - 15/02	16/02 -22/02	23/02 - 01/03	02/03 - 08/03	09/03 - 15/03	16/03 - 22/03	23/03 - 29/03	30/03 - 05/04	06/04 - 12/04	13/04 - 19/04	20/04 - 26/04	27/04 - 03/05	04/05 - 10/05	11/05 - 17/05	18/05 - 24/05	25/05 - 31/05	01/06 - 07/06	08/06 - 14/06	15/06 - 21/06	22/06 - 26/06
Literature Review																						
System Design																						
Core Implemetation																						
Reporting and Interface																						
Testing and evaluation																						
Thesis writing																						
		Low Risk																				
		Med Risk																				
		High Risk																				

Figure 1: Time Table

Low risk: Delays during the initial literature review phase are unlikely to significantly impact the overall timeline. However, the literature review process is inherently iterative and will continue throughout the project. In particular, the exploration of technical documentation (e.g., framework internals and data loading mechanisms) must be conducted concurrently with the implementation phase. While this may introduce minor delays, the risk is considered low as these investigations are tightly coupled with system design and are unlikely to block progress. The same goes for thesis writing, it is by nature an iterative process and should not pose any delays if it is done concurrently with other phases.

Medium risk: The system design and architectural phase poses a moderate risk. Design decisions made early in the project may influence later implementation complexity, and unforeseen technical constraints could require revisions. Additionally, the testing and evaluation phase depends on clearly defined experimental setups and metrics. But it should not only be done on the tail end, some forms of benchmarks and correctness checks should be done in parallel with implementation. Insufficient planning in earlier phases could lead to timeline adjustments.

High risk: The core implementation phase presents the highest risk. Challenges related to instrumenting dataset access within existing ML frameworks (e.g. Pytorch), handling randomized data shuffling, batching, and multiprocessing within standard ML data loaders, and ensuring acceptable runtime overhead may introduce unforeseen technical difficulties. The reporting and CLI interface also poses high risk, as they must transform the raw provenance logs into meaningful summaries without unnecessary complexity. Early prototyping and incremental validation will be used to mitigate this risk.

Some notable references that have been gathered and read [10, 11, 12, 13] .

References

[1] OECD, *Progress in Implementing the European Union Coordinated Plan on Artificial Intelligence (Volume 2): Uptake in High-Impact Sectors*. Paris: OECD Publishing, 2026. [Online]. Available: https://doi.org/10.1787/3ac96d41-en

[2] OECD and Financial Stability Board, "Oecd–fsb roundtable on artificial intelligence (ai) in finance: Summary of key findings," https://www.oecd.org/content/dam/oecd/en/topics/policy-sub-issues/digital-finance/OECD%20%E2%80%93%20FSB%20Roundtable%20on%20Artificial%20Intelligence%20(AI)%20in%20Finance.pdf, 2024, oECD Publishing, Paris. Accessed: 2026-02-23.

[3] OECD, *Governing with Artificial Intelligence: The State of Play and Way Forward in Core Government Functions.* Paris: OECD Publishing, 2025. [Online]. Available: https://doi.org/10.1787/795de142-en

[4] European Parliament and Council of the European Union, "Regulation (eu) 2024/1689 of the european parliament and of the council of 13 june 2024 laying down harmonised rules on artificial intelligence (artificial intelligence act)," 2024, oJ L, 12.7.2024.

[5] (2026) Pytorch documentation. Official documentation for the PyTorch open source machine learning library. [Online]. Available: https://docs.pytorch.org/docs/stable/index.html

[6] (2026) Weights & biases documentation. Technical documentation for the Weights & Biases platform. [Online]. Available: https://docs.wandb.ai/

[7] (2025) Mlflow: A tool for managing the machine learning lifecycle. Official machine learning documentation for the MLflow platform. [Online]. Available: https://mlflow.org/docs/latest/ml/

[8] (2026) Dvc documentation. Official documentation for Data Version Control (DVC). [Online]. Available: https://doc.dvc.org/

[9] E. Semenova, A. Sheinkman, T. J. Hitge, S. M. Hall, and J. Cockayne, "Smrs: advocating a unified reporting standard for surrogate models in the artificial intelligence era," 2025. [Online]. Available: https://arxiv.org/abs/2502.06753

[10] L. Houston, Y. Probst, and A. Martin, "Assessing data quality and the variability of source data verification auditing methods in clinical research settings," *Journal of Biomedical Informatics*, vol. 83, pp. 25–32, 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1532046418300935

[11] B. Marino, M. Kurmanji, and N. D. Lane, "Position: Bridge the gaps between machine unlearning and ai regulation," 2025. [Online]. Available: https://arxiv.org/abs/2502.12430

[12] X. Mu, M. Pang, and F. Zhu, "Data provenance via differential auditing," 2022. [Online]. Available: https://arxiv.org/abs/2209.01538

[13] A. Wernick and K. Meding, "Beware! the ai act can also apply to your ai research practices," 2025. [Online]. Available: https://arxiv.org/abs/2506.03218